Pentru examen:

- greedy: 3p (pt ca e gresit)
- back: 5p (pt ca e ineficient)
- DP (oricare dintre solutii): 9p

Gigel s-a lasat de furat obiecte, de batut cuie in garduri sau de stropitori. Acum isi propune sa treaca de la categoria pana la categoria grea la box. Pentru asta trebuie sa puna multe kilograme le el. A aflat de la un prieten care mananca ciocolata toata ziua, ca ciocolata poate ingrasa. Gigel doreste sa isi rezolve problema mancand ciocolata!

El a pus pe masa **n** ciocolate (albe sau negre), numerotate 1, 2, 3, ..., n.
Pentru ca a cumparat prea multa ciocolata, acum vrea sa creeze un joc.

Are voie sa manance ce ciocolate vrea, dar cu cateva restrictii:
- daca a mancat ciocolata **i** si aceasta este **NEAGRA**, **NU** are voie sa manance ciocolata **i + 1**
- daca a mancat ciocolata **i** si aceasta este **ALBA**, **NU** are voie sa manance ciocolatele **i + 1** si **i + 2.**

Fiecare ciocolata i are asociata o pereche **(c[i], t[i])**, unde:
c[i] = numarul de calorii pe care le contine ciocolata i
t[i] = tipul ciocolatei i (0/1 - ALBA/NEAGRA)

Ajutati-l pe Gigel sa asimileze o **CANTITATE MAXIMA DE CALORII**, respectand regulile de mai sus.

(solutii mai jos - scroll pana la final)

Ar trebui sa intre orice dinamica O(n) (timp + spatiu) corecta.

Solutie 1:

dp[i] = numarul maxim de calorii pe care Gigel il poate asimila mancand din c[i..n]

dp[n] = c[n]
dp[n - 1] = max(c[n], c[n - 1])
dp[n - 2] = max(new_sol, dp[n - 1])

new_sol poate avea UNA dintre urmatoare valori
1. c[n - 2] + dp[n], daca tip[n - 2] != ALBA (nu am voie n - 1)
2. c[n - 2] , daca tip[n - 2] == ALBA (nu am voie n - 1 si n)

// EAT c[i]? YES / NO
dp[i] = max(new_sol, dp[i + 1]), i = n - 3:1

new_sol poate avea UNA dintre urmatoare valori
1. c[i] + dp[i + 2], daca tip[i] != ALBA (NU am voie i + 1)
2. c[i] + dp[i + 3], daca tip[i] == ALBA (NU am voie i + 1 si i + 2),

R: dp[1] (numarul maxim de calorii din 1..n)

Solutie 2:

dp[i] = numarul maxim de calorii pe care Gigel il poate asimila mancand din c[1..i]

dp[1] = c[1] // o varianta
dp[2] = max(c[1], c[2]) // pe una dintre ele
dp[3] = max(new_sol, dp[2])

new_sol are UNA dintre valorile
1. dp[1] + c[3] daca tip[1] != ALBA
2. c[3] daca tip[1] == ALBA

(Se poate scapa de dp[3] caz special daca se face dp[0] = 0)

// EAT c[i]? YES / NO
dp[i] = max(new_sol, dp[i - 1]) i = 3:n

new_sol are UNA dintre urmatoare valori
1. dp[i - 2] + c[i], daca tip[i - 2] != ALBA
2. dp[i - 3] + c[i], daca tip[i - 2] == ALBA

R: dp[n] (numarul maxim de calorii din 1..n)

Solutie 3

dp[i][0] = numarul maxim de calorii pe care Gigel il poate asimila mancand din c[1..i] si NU MANANCA ciocolata i

dp[i][1] = numarul maxim de calorii pe care Gigel il poate asimila mancand din c[1..i] si MANANCA ciocolata i

Solutie 4: Ca si solutia 2, doar ca are restrictia ca dp[i] cuprinde cea mai buna sol care mananca ciocolata i